

Yesterday 3:07 PM



Checked vs unchecked exceptic Free offer



Exception vs Error.

throw vs throws.

Can we have multiple catch blocks?

How does Java determine which catch block to execute?

Can catch block come before try?

Can finally execute without catch?

Can finally execute if return is used?

What happens here?

```
try {  
    return 10;
```

Show more ▾

Absolutely. These are **very common Java interview questions**, especially around 4–5 years of experience. I'll explain them in an interview-friendly way, including the tricky `finally + return` case.

1. Checked vs Unchecked Exceptions

Checked Exception

Checked exceptions are checked by the **compiler at compile time**.

You must either:

1. Handle them using `try-catch` , or
2. Declare them using `throws` .

Examples:

Java



```
IOException  
SQLException
```

`FileNotFoundException`

Example:

```
</> Java

public void readFile() throws IOException {
    FileReader reader = new FileReader("test.txt");
}
```

If you don't handle or declare it, compilation fails.

Unchecked Exception

Unchecked exceptions are **not checked by the compiler**.

They are subclasses of `RuntimeException`.

Examples:

```
</> Java

NullPointerException
ArithmeticException
ArrayIndexOutOfBoundsException
IllegalArgumentException
```

Example:

```
</> Java

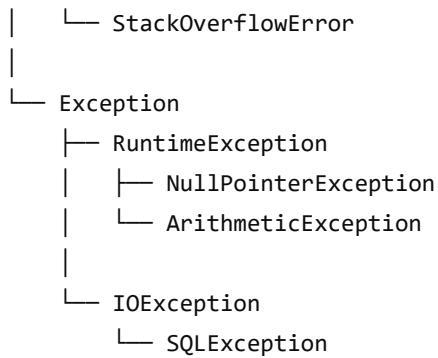
int result = 10 / 0; // ArithmeticException at runtime
```

Interview answer

Checked exceptions are compile-time checked and must be either caught or declared. Unchecked exceptions occur at runtime and are subclasses of `RuntimeException`.

Hierarchy

```
Throwable
├── Error
│   └── OutOfMemoryError
```



2. Exception vs Error

Both `Exception` and `Error` extend `Throwable`.

Exception

An exception generally represents a condition that an application **can reasonably handle**.

Examples:

</> **Java**



```

IOException
SQLException
IllegalArgumentException
NullPointerException
  
```

Error

An `Error` generally represents a serious JVM/system-level problem that applications normally **should not try to recover from**.

Examples:

</> **Java**



```

OutOfMemoryError
StackOverflowError
NoClassDefFoundError
  
```

For example:

</> **Java**



```
public void recursion() {  
    recursion();  
}
```

Eventually this can cause:

StackOverflowError



Interview answer

Exceptions represent conditions that application code may handle, whereas Errors generally indicate serious JVM or system problems that applications normally shouldn't attempt to recover from.

3. throw VS throws

This is a very common interview question.

throw

`throw` is used to **actually throw an exception object**.

</> Java



```
throw new IllegalArgumentException("Age cannot be negative");
```

It is used inside method/block code.

throws

`throws` is used in a method declaration to say:

"This method may throw these exceptions; the caller is responsible for handling them."

</> Java



```
public void readFile() throws IOException {  
    // code  
}
```

Difference

throw	throws
Actually throws exception	Declares possible exceptions
Used inside method	Used in method signature
Throws one exception object at a time	Can declare multiple exceptions
<code>throw new IOException();</code>	<code>throws IOException</code>

Example:

</> **Java**



```
public void process() throws IOException {

    if (someCondition) {
        throw new IOException("File problem");
    }

}
```

Here:

- `throws` → declares
- `throw` → actually throws

4. Can we have multiple catch blocks?

Yes.

Example:

</> **Java**



```
try {
    int result = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Arithmetic problem");
}
catch (RuntimeException e) {
    System.out.println("Runtime problem");
}
```

```
}  
catch (Exception e) {  
    System.out.println("General exception");  
}
```

This is valid because the exceptions are arranged from **more specific** → **more general**.

5. How does Java determine which catch block to execute?

This is important.

Java checks the `catch` blocks **from top to bottom**.

Suppose:

```
</> Java  
  
try {  
    int x = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Arithmetic");  
}  
catch (RuntimeException e) {  
    System.out.println("Runtime");  
}  
catch (Exception e) {  
    System.out.println("Exception");  
}
```

The thrown exception is:

ArithmeticException

Java checks:

ArithmeticException → MATCH ✓

So it executes the first catch and **doesn't check the remaining catches**.

Important rule

The more specific exception must come **before** the more general exception.

Correct:

```
</> Java

catch (ArithmeticException e) {}
catch (RuntimeException e) {}
catch (Exception e) {}
```



Wrong:

```
</> Java

catch (Exception e) {}
catch (ArithmeticException e) {} // compilation error
```



Why?

Because `Exception` can already catch `ArithmeticException`.

So the second catch becomes **unreachable**.

6. Can catch block come before try?

No.

This is invalid:

```
</> Java

catch (Exception e) {

}

try {

}
```



A `catch` must immediately follow its associated `try` block.

Valid:

```
</> Java
```



```
try {  
  
}  
catch (Exception e) {  
  
}
```

You can also have:

```
</> Java  
  
try {  
  
}  
finally {  
  
}
```

or:

```
</> Java  
  
try {  
  
}  
catch (Exception e) {  
  
}  
finally {  
  
}
```

But a standalone `catch` is not allowed.

7. Can `finally` execute without `catch`?

Yes.

A `try` must be followed by at least one `catch` **or** a `finally`.

So this is completely valid:

```
</> Java
```

```
try {  
    System.out.println("Hello");  
}  
finally {  
    System.out.println("Finally");  
}
```

Output:

```
Hello  
Finally
```



This is useful when you want cleanup but don't want to handle the exception at that point.

For example:

</> **Java**



```
try {  
    // resource operation  
}  
finally {  
    // cleanup  
}
```

8. Can finally execute if return is used?

Yes.

This is one of the most important interview points.

Consider:

</> **Java**



```
public int test() {  
  
    try {  
        return 10;  
    }  
    finally {  
        System.out.println("Finally");  
    }  
}
```

```
}  
}
```

Output:

Finally



And the method returns:

10



What happens internally conceptually?

When Java reaches:

```
</> Java  
  
return 10;
```



the method prepares to return 10 .

But before the method actually completes, Java executes:

```
</> Java  
  
finally
```



Then it returns 10 .

Think:

```
try  
↓  
return value prepared  
↓  
finally executes  
↓  
method returns
```



9. The tricky one

You asked:

</> Java



```
try {  
    return 10;  
} finally {  
    return 20;  
}
```

Answer:

20



The `finally` return **overrides the try return**.

Conceptually:

```
try:  
    return 10  
    ↓  
finally executes  
    ↓  
return 20  
    ↓  
20 is returned
```



So:

</> Java



```
public int test() {  
    try {  
        return 10;  
    } finally {  
        return 20;  
    }  
}
```

returns:

20

**Why?**

Because the `return` inside `finally` causes the method to complete with `20`, replacing the pending return from `try`.



Interview trap: Never use `return` in `finally`

Although Java allows it, it's considered **bad practice** because it can suppress:

- a return value from `try`
- an exception thrown from `try`
- an exception thrown from `catch`

For example:

 **Java**



```
public int test() {  
  
    try {  
        throw new RuntimeException("Problem");  
    }  
    finally {  
        return 20;  
    }  
}
```

What happens?

The exception is effectively suppressed by the `return 20`.

The method returns:

20



instead of propagating the exception.

So in production code:

Avoid returning from `finally`.



One important `finally` exception

`finally` normally executes even when there is a `return` , but there are situations where it may not execute, such as when the JVM is forcibly terminated.

For example:

```
</> Java

try {
    System.out.println("Try");
}
finally {
    System.out.println("Finally");
}
```

Normally `finally` executes.

But something like:

```
</> Java

System.exit(0);
```

can terminate the JVM before normal completion of the program.

Interview-ready summary

If the interviewer asks these rapidly, you can answer:

Question	Short answer
Checked vs unchecked?	Checked at compile time; unchecked are runtime exceptions
Exception vs Error?	Exception can generally be handled; Error indicates serious JVM/system problems
throw vs throws ?	throw actually throws; throws declares possible exceptions
Multiple catch?	Yes
Which catch executes?	First matching catch from top to bottom

Question	Short answer
Specific catch before general?	Yes
Can catch come before try?	No
Can finally exist without catch?	Yes
Can finally execute after return?	Yes
<code>return</code> in try + <code>return</code> in finally?	Finally's return wins
Should we return from finally?	No, generally avoid it because it can suppress returns/exceptions

The 3 rules I'd especially remember for a 5-year Java interview:

Rule 1:

Specific catch → General catch



Rule 2:

`try` + `finally` is valid



Rule 3:

`finally` executes before normal method completion,
but a `return` inside `finally` can override the earlier `return`.



Why should we avoid returning from finally?

What happens if an exception occurs inside finally?

Can we throw checked exceptions from a method without declaring them?

Can an overriding method throw a broader checked exception?

Can an overriding method throw a narrower checked

exception?

Can we catch Exception before IOException?

What is try-with-resources?

What is AutoCloseable?

Show more ▾

These are **very good 5-year Java interview questions** because they test whether you understand not just syntax, but **exception propagation, inheritance, and API design**.

1. Why should we avoid returning from `finally`?

Because it can **override the return value or suppress an exception**.

Example 1 — overrides return

</> Java

```
public int test() {  
    try {  
        return 10;  
    } finally {  
        return 20;  
    }  
}
```



Result:

20



The `finally` return wins.

Example 2 — suppresses exception

</> Java

```
public void test() {  
    try {  
        throw new RuntimeException("Something went wrong");  
    } finally {  
        return;  
    }  
}
```



```
}  
}
```

Normally, the `RuntimeException` should propagate.

But the `return` in `finally` causes the method to complete normally, so the exception is effectively **suppressed**.

Interview answer

We should avoid returning from `finally` because it can override a pending return value and, more seriously, suppress exceptions that should have propagated.

2. What happens if an exception occurs inside `finally`?

An exception thrown from `finally` can **replace/suppress** the exception that was already **in flight**.

Consider:

```
</> Java  
  
public void test() {  
    try {  
        throw new RuntimeException("Exception from try");  
    } finally {  
        throw new RuntimeException("Exception from finally");  
    }  
}
```

The caller sees:

```
Exception from finally
```

The exception from `try` is no longer the primary propagated exception.

Another case

```
</> Java  
  
try {  
    // normal code
```

```
}  
finally {  
    throw new RuntimeException("Finally failed");  
}
```

The exception from `finally` propagates to the caller.

Important interview point

This is one reason cleanup code inside `finally` should be written carefully.

With **try-with-resources**, Java has special handling for this situation through **suppressed exceptions**, which we'll see below.

3. Can we throw checked exceptions from a method without declaring them?

Normally: No.

If you explicitly throw a checked exception:

```
</> Java   
  
public void read() {  
    throw new IOException();  
}
```

This doesn't compile because `IOException` is checked.

You need:

```
</> Java   
  
public void read() throws IOException {  
    throw new IOException();  
}
```

or handle it:

```
</> Java   
  
public void read() {  
    try {
```

```
        throw new IOException();  
    } catch (IOException e) {  
        // handle  
    }  
}
```

But there is a famous advanced exception: "sneaky throw"

Java can technically throw a checked exception without declaring it using generic/type-inference tricks or libraries.

But that's an **advanced technique and generally not appropriate for normal application code**.

For a 5-year interview, the safe answer is:

A checked exception normally must be caught or declared with `throws`. There are advanced "sneaky throw" techniques, but they bypass the compiler's checked-exception mechanism and should be used cautiously.

4. Can an overriding method throw a broader checked exception?

✗ No.

Suppose parent:

```
</> Java   
  
class Parent {  
    public void process() throws IOException {  
    }  
}
```

Child cannot do:

```
</> Java   
  
class Child extends Parent {  
    public void process() throws Exception { // ✗  
    }
```

```
}  
}
```

Why?

Because `Exception` is broader than `IOException`.

Imagine:

</> **Java**



```
Parent p = new Child();  
p.process();
```

The caller only knows that `Parent.process()` throws `IOException`.

Allowing the child to suddenly throw `Exception` would break the parent's contract.

Rule

An overriding method cannot throw a broader checked exception than the overridden method.

5. Can an overriding method throw a narrower checked exception?

✓ Yes.

Parent:

</> **Java**



```
class Parent {  
    public void process() throws Exception {  
    }  
}
```

Child:

</> **Java**



```
class Child extends Parent {
    @Override
    public void process() throws IOException {
    }
}
```

Valid.

Because:

```
IOException
  ↓
Exception
```



IOException is narrower.

It can also throw no checked exception

</> Java



```
class Child extends Parent {
    @Override
    public void process() {
    }
}
```

Also valid.

So:

Parent:
throws Exception



Child:

- throws IOException ✓
- throws FileNotFoundException ✓
- throws nothing ✓
- throws RuntimeException ✓

Child:
throws SQLException? ✓

Wait — why is SQLException also allowed?

Because both IOException and SQLException are subclasses of Exception .

The child can throw a different **narrower checked exception**, as long as it is a subtype of the parent's declared checked exception.

But:

```
</> Java
```



```
throws Error
```

doesn't follow the checked-exception restriction because `Error` is unchecked.

6. Can we catch `Exception` before `IOException`?

✗ No.

This is invalid:

```
</> Java
```



```
try {  
    // code  
}  
catch (Exception e) {  
}  
catch (IOException e) {  
}
```

Why?

Because:

```
Exception  
  ↑  
IOException
```



`Exception` already catches `IOException`.

Therefore the second catch becomes **unreachable**.

Correct

```
</> Java
```



```
try {  
    // code  
}  
catch (IOException e) {  
}  
catch (Exception e) {  
}
```

Think:

Specific
↓
General



7. What is try-with-resources?

Try-with-resources is a Java feature for automatically closing resources.

Before Java 7, you might write:

```
</> Java  
  
FileInputStream input = null;  
  
try {  
    input = new FileInputStream("data.txt");  
  
    // use resource  
  
} finally {  
    if (input != null) {  
        input.close();  
    }  
}
```



You have to manually close the resource.

With try-with-resources:

```
</> Java  
  
try (FileInputStream input = new FileInputStream("data.txt")) {  
  
    // use input
```



```
}
```

Java automatically closes `input` .

Multiple resources

```
</> Java
```



```
try (  
    FileInputStream input = new FileInputStream("input.txt");  
    FileOutputStream output = new FileOutputStream("output.txt")  
) {  
  
    // use resources  
  
}
```

Both are automatically closed.

Closing order

Resources are closed in **reverse order of creation**.

```
input created  
output created  
  
output closed first  
input closed second
```



8. What is AutoCloseable ?

`AutoCloseable` is an interface that allows an object to be used with **try-with-resources**.

Simplified:

```
</> Java
```



```
public interface AutoCloseable {  
    void close() throws Exception;  
}
```

Example:

</> **Java**

```
class MyResource implements AutoCloseable {

    public void use() {
        System.out.println("Using resource");
    }

    @Override
    public void close() {
        System.out.println("Closing resource");
    }
}
```

Now:

</> **Java**

```
try (MyResource resource = new MyResource()) {
    resource.use();
}
```

Output conceptually:

```
Using resource
Closing resource
```



The JVM/compiler-generated resource-management logic ensures `close()` is invoked when leaving the try-with-resources statement.

Closeable VS AutoCloseable

`Closeable` extends `AutoCloseable` and is commonly used for I/O resources.

AutoCloseable



Closeable



`AutoCloseable.close()` can declare `Exception`, while `Closeable.close()` declares `IOException`.

9. What is a suppressed exception?

This is a **very important try-with-resources** interview question.

Suppose:

```
</> Java

try (MyResource resource = new MyResource()) {
    throw new RuntimeException("Main exception");
}
```

Then while closing the resource:

```
</> Java

resource.close()
```

also throws an exception.

Now we have two exceptions:

```
Exception from try
Exception from close()
```

Which one should be reported as the main exception?

Java treats the exception from the `try` block as the **primary exception** and the exception from `close()` as a **suppressed exception**.

Conceptually:

```
Primary exception
|
+---- suppressed exception
```

You can access suppressed exceptions:

```
</> Java

catch (Exception e) {

    for (Throwable suppressed : e.getSuppressed()) {
        System.out.println(suppressed);
    }
}
```

Why is this useful?

Without suppressed exceptions, the exception from `close()` could hide the more important original exception.

Try-with-resources preserves both.

10. How would you create a custom exception?

There are two common choices.

Custom unchecked exception

Extend `RuntimeException`:

</> Java



```
public class InvalidOrderException extends RuntimeException {  
  
    public InvalidOrderException(String message) {  
        super(message);  
    }  
}
```

Use:

</> Java



```
if (order == null) {  
    throw new InvalidOrderException("Order cannot be null");  
}
```

Custom checked exception

Extend `Exception`:

</> Java



```
public class PaymentException extends Exception {  
  
    public PaymentException(String message) {  
        super(message);  
    }  
}
```

```
}  
}
```

Use:

</> Java



```
public void processPayment() throws PaymentException {  
  
    if (/* payment failed */) {  
        throw new PaymentException("Payment failed");  
    }  
}
```

With cause

A good production-level custom exception often supports a cause:

</> Java



```
public class PaymentException extends RuntimeException {  
  
    public PaymentException(String message, Throwable cause) {  
        super(message, cause);  
    }  
}
```

Then:

</> Java



```
try {  
    // database call  
}  
catch (SQLException e) {  
    throw new PaymentException("Unable to process payment", e);  
}
```

This preserves the original exception as the **cause**.

11. When should you use checked vs unchecked exceptions?

This is more of an **API/design question**, so interviewers like to hear reasoning rather than a rigid rule.

Use checked exception when:

The caller can **reasonably recover from the condition** and you want to make handling part of the API contract.

Examples:

```
File not found  
Database operation failure  
Network/resource operation failure
```



For example:

```
</> Java  
  
public void readFile() throws IOException
```



The caller may reasonably decide:

```
File missing?  
→ ask for another file  
→ use fallback  
→ show appropriate message
```



Use unchecked exception when:

The problem is usually a **programming error, invalid state, or invalid argument**, or when forcing every caller to handle it doesn't provide much value.

Examples:

```
NullPointerException  
IllegalArgumentException  
IllegalStateException  
IndexOutOfBoundsException
```



Custom business exceptions are often unchecked in modern Java applications, depending on the application's error-handling architecture.

Example:

</> Java



```
public void withdraw(double amount) {  
  
    if (amount <= 0) {  
        throw new IllegalArgumentException("Amount must be positive");  
    }  
}
```

★ A strong 5-year interview answer

If the interviewer asks:

"How do you decide between checked and unchecked exceptions?"

You can say:

"I prefer checked exceptions when the caller can reasonably recover from the condition and handling it is an important part of the API contract. I prefer unchecked exceptions for programming errors, invalid arguments, invalid state, and business/application exceptions where centralized exception handling is more appropriate. I also avoid excessive checked exceptions because they can make APIs noisy and force callers to handle exceptions they cannot meaningfully recover from."

🔥 Final interview cheat sheet

Question

Answer

Return from `finally` ?

Avoid it; it can override return and suppress exceptions

Exception in `finally` ?

It can replace the exception already being propagated

Checked exception without `throws` ?

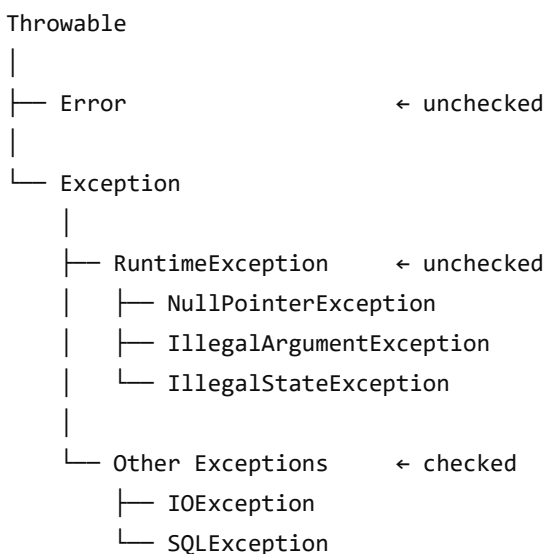
Normally no; must catch or declare

Override with broader checked exception?

✗ No

Question	Answer
Override with narrower checked exception?	✓ Yes
Override with no checked exception?	✓ Yes
catch(Exception) before catch(IOException) ?	✗ No
Try-with-resources?	Automatically closes resources
AutoCloseable ?	Interface whose resources can be used in try-with-resources
Suppressed exception?	Secondary exception preserved when primary exception occurs
Custom exception?	Extend Exception OR RuntimeException
Checked exception?	When caller can reasonably recover and handling is part of contract
Unchecked exception?	Programming errors, invalid state/arguments, or cases suited to centralized handling

One hierarchy to remember



And the **most important overriding rule**:

Parent: throws Exception



Child:

throws IOException ✓
 throws FileNotFoundException ✓
 throws nothing ✓
 throws RuntimeException ✓
 throws SQLException ✓
 throws broader checked exception ✗



Should business exceptions be checked or unchecked?
 How do you design exception handling in a REST API?
 How do you prevent sensitive information from appearing
 in stack traces returned to clients?
 Production scenario

Your API calls three downstream services. One service
 throws a timeout, another returns 500, and the third
 succeeds. How should your exception handling work?

For a 5-year Java/Spring Boot interview, these questions are really about **exception design + API architecture**, not just `try/catch`.

1. Should business exceptions be checked or unchecked?

Usually: Unchecked (RuntimeException)

For modern Spring Boot REST applications, business/domain exceptions are commonly represented as unchecked exceptions.

Example:

</> Java



```
public class InsufficientBalanceException extends RuntimeException {  
  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}
```

Then:

</> Java



```
if (account.getBalance() < amount) {  
    throw new InsufficientBalanceException("Insufficient balance");  
}
```

Why unchecked?

Because business exceptions generally don't represent a condition where **every caller must recover locally**.

Instead, you can handle them centrally:

</> Java



```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(InsufficientBalanceException.class)  
    public ResponseEntity<?> handle(InsufficientBalanceException ex) {  
        // create API response  
    }  
}
```

So your service code remains clean.

When could checked make sense?

If your application specifically wants to **force callers to acknowledge/recover from a condition**, checked exceptions can be appropriate.

But don't say:

"Business exceptions must always be unchecked."

That's too absolute.

Strong interview answer

"I generally prefer business exceptions as unchecked exceptions in Spring Boot applications because they represent domain-level failures rather than conditions that every caller can meaningfully recover from. I handle them centrally using `@RestControllerAdvice`. I would use checked exceptions when forcing callers to explicitly handle a recoverable condition is genuinely valuable."

2. How do you design exception handling in a REST API?

A good design is:

```
graph TD; Controller --> Service; Service --> Repository["Repository / Downstream service"]; Repository --> Exception; Exception --> GlobalHandler["Global Exception Handler"]; GlobalHandler --> Response["Consistent HTTP response"];
```

Don't put this everywhere:

```
<> Java

try {
    // business logic
}
catch (Exception e) {
    return ResponseEntity.status(500)...
}
```

Instead, let meaningful exceptions propagate and handle them centrally.

Step 1: Define application exceptions

For example:

</> Java



```
public class OrderNotFoundException extends RuntimeException {  
    public OrderNotFoundException(String message) {  
        super(message);  
    }  
}
```

</> Java



```
public class PaymentFailedException extends RuntimeException {  
    public PaymentFailedException(String message, Throwable cause) {  
        super(message, cause);  
    }  
}
```

Step 2: Throw them from service layer

</> Java



```
public Order getOrder(Long id) {  
  
    Order order = repository.findById(id)  
        .orElseThrow(() ->  
            new OrderNotFoundException("Order not found"));  
  
    return order;  
}
```

Step 3: Handle globally

Spring provides `@RestControllerAdvice` for centralized REST exception handling.

</> Java



```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(OrderNotFoundException.class)  
    public ResponseEntity<ApiError> handleNotFound(  
        OrderNotFoundException ex) {
```

```
        ApiError error = new ApiError(  
            "ORDER_NOT_FOUND",  
            ex.getMessage()  
        );  
  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(error);  
    }  
}
```

Step 4: Return a consistent error structure

For example:

</> JSON



```
{  
  "code": "ORDER_NOT_FOUND",  
  "message": "Order not found",  
  "timestamp": "2026-09-01T15:00:00Z",  
  "traceId": "abc-123"  
}
```

The important thing is that clients don't receive random error formats from different controllers.

3. How do you prevent sensitive information from appearing in stack traces returned to clients?

This is very important in production.

Never do this:

</> Java



```
@ExceptionHandler(Exception.class)  
public ResponseEntity<> handle(Exception e) {  
  
    return ResponseEntity  
        .status(500)
```

```
.body(e.getStackTrace());  
}
```

Or:

```
</> Java  
  
return new ResponseEntity<>(e.getMessage(), ...);
```

for arbitrary internal exceptions.

Because the exception message might contain:

```
database hostname  
SQL query  
table names  
file paths  
internal service URLs  
tokens  
user information  
internal implementation details
```

Better approach

Client receives:

```
</> JSON  
  
{  
  "code": "INTERNAL_ERROR",  
  "message": "Something went wrong",  
  "traceId": "7f82ab"  
}
```

Server logs:

```
ERROR Payment failed  
traceId=7f82ab  
  
java.sql.SQLException: connection refused...  
    at ...  
    at ...
```

So:

CLIENT



Safe error response

SERVER



Detailed logs



Use correlation/trace IDs

For example:

Request



X-Request-ID: abc123



API



Logs



Response:

`</>` JSON

```
{
  "code": "INTERNAL_ERROR",
  "message": "Unexpected error",
  "traceId": "abc123"
}
```



Now the customer/support team can give you:

```
traceId = abc123
```



and you can find the detailed error in your logs.

Important

Also be careful with logging.

This is dangerous:

</> Java



```
log.error("Payment request: {}", paymentRequest);
```

if `paymentRequest` contains sensitive information.

Don't log:

```
passwords  
access tokens  
credit-card information  
session credentials  
personal secrets
```



Use structured logging and sanitization/redaction where appropriate.

4. Production scenario

Your API calls three downstream services.

Service A → timeout

Service B → HTTP 500

Service C → succeeds

How should exception handling work?

This is where a **5-year candidate** should think about business requirements first.

The first question shouldn't be:

"Which exception should I catch?"

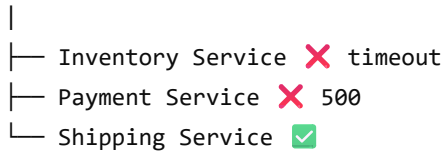
It should be:

"Are these three downstream calls independent or mandatory for the final operation?"

Case 1: All three services are mandatory

Suppose:

Order API



If the API cannot produce a valid response without A and B, you shouldn't return success just because C succeeded.

You need to propagate an appropriate failure.

For example:

API

↓

Dependency failure

↓

503 Service Unavailable



Potential response:

</> JSON

```
{
  "code": "DEPENDENCY_FAILURE",
  "message": "Unable to complete the request",
  "traceId": "abc123"
}
```



Don't tell the client:

"Inventory service timed out at 10.20.1.45"



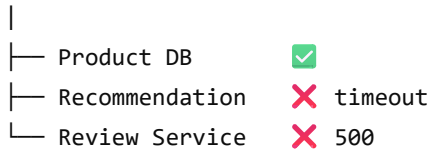
That's internal implementation information.

Case 2: Services are independent / optional

This is different.

Suppose:

Product API



Recommendations and reviews aren't necessary to return the product.

Then you could return:

</> JSON

```
{
  "product": {
    "id": 101,
    "name": "Phone"
  },
  "recommendations": [],
  "reviews": []
}
```

with appropriate internal logging/metrics.

This is called **partial degradation / graceful degradation**.

The key principle:

A failure in an optional dependency shouldn't necessarily fail the entire request.

Case 3: Independent calls can run in parallel

If the three services are independent, don't necessarily do:

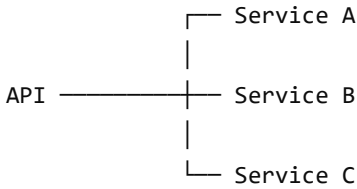
A
↓
B
↓
C

That gives you:

Total latency $\approx A + B + C$



Instead:



Run them concurrently.

Then:

Total latency $\approx \max(A, B, C)$



plus overhead.

In Java/Spring Boot, depending on the workload, you could use:

- `CompletableFuture`
- appropriate task executors
- virtual threads for suitable blocking I/O workloads
- reactive programming when the architecture calls for it

Case 4: Timeout vs 500 should be treated differently

This is another good interview point.

Timeout

```
Service A
  ↓
No response within timeout
  ↓
TimeoutException
```



Potential response:

503 Service Unavailable



or graceful degradation if the dependency is optional.

You should also have:

```
timeout
+
retry (carefully)
+
circuit breaker
+
fallback
```



where appropriate.

Downstream 500

```
Service B
↓
HTTP 500
↓
Downstream service failure
```



Usually don't blindly retry every 500.

You need to consider:

- Is the operation idempotent?
- Is the failure transient?
- Could retry make the problem worse?
- Is the dependency already overloaded?

For example, a retry policy might be:

```
Request
↓
500
↓
Retry with backoff
↓
500
↓
Circuit breaker / fallback
```



But only where retrying makes sense.

5. What if one service succeeds and another fails?

This is where **partial success vs atomicity** matters.

Imagine:

Order Service	✓
Payment Service	✗
Inventory Service	✗



If these operations modify state, you can't simply say:

"Two failed, so I'll throw an exception."

You need to consider whether previous successful operations need compensation.

For distributed transactions, patterns such as **Saga / compensating transactions** may be appropriate.

For example:

```
Create Order
  ↓
Reserve Inventory
  ↓
Charge Payment
```



If:

Create Order	✓
Reserve Inventory	✓
Charge Payment	✗



you might need:

```
Payment failed
  ↓
Release inventory
  ↓
Mark order as PAYMENT_FAILED
```



That's very different from merely catching an exception.

★ How I'd answer this scenario in an interview

You can give this answer almost directly:

"First, I'd determine whether the three downstream calls are mandatory or optional for the business operation. If they are optional, I'd use graceful degradation and return the successful data while representing unavailable sections appropriately. If they are mandatory, I wouldn't return success just because one dependency succeeded; I'd return an appropriate failure, usually 503 for a dependency availability problem, while keeping internal details out of the response.

I would handle timeouts and downstream 5xx errors separately, with bounded timeouts, carefully configured retries with backoff where appropriate, and circuit breakers to prevent cascading failures. If the calls are independent, I'd execute them concurrently to reduce latency.

Finally, if the calls involve state changes, I'd consider consistency and compensation. A successful downstream operation followed by a failed operation may require a Saga or compensating action rather than simply propagating the exception."

That is a **much stronger 5-year answer** than simply saying:

Java

```
try {  
    callA();  
    callB();  
    callC();  
}  
catch (Exception e) {  
    return 500;  
}
```

Because in production, **exception handling is really about failure semantics, resilience, consistency, observability, and what the client is allowed to know.**



ChatGPT can make mistakes. Check important info.